

Lettura dei dati

Alias

Come usare AS per dare soprannomi temporanei a colonne e tabelle.

A volte un nome più corto o più chiaro aiuta tantissimo

Un alias è un soprannome temporaneo. Lo usi per rendere i risultati più leggibili o per abbreviare i nomi nelle query più lunghe.

```
SELECT nome AS cliente
FROM clienti;
```

Qui la colonna `nome` apparirà con il nome `cliente` nel risultato.

Perché sono comodi

Gli alias aiutano in due modi:

- rendono il risultato più parlante
- accorciano la scrittura quando ci sono tabelle con nomi lunghi

Alias sulle tabelle

Gli alias non valgono solo per le colonne. Puoi usarli anche sulle tabelle:

```
SELECT c.nome
FROM clienti AS c;
```

Qui `c` è un soprannome della tabella `clienti`.

Dove diventano davvero preziosi

Nelle query con `JOIN`, gli alias migliorano tantissimo la leggibilità. Invece di ripetere sempre il nome completo della tabella, puoi usare un nome corto e chiaro.

Non cambiano i dati. Cambiano solo il modo in cui la query viene scritta o mostrata.

Operatori di confronto

Uguale, maggiore, minore, BETWEEN, LIKE e IN spiegati con esempi chiari.

Sono i simboli con cui SQL confronta i valori

Gli operatori di confronto dicono al database come confrontare i valori. Senza di loro, SQL non saprebbe se una riga è abbastanza grande, abbastanza piccola, uguale a ciò che stai cercando o compresa in un elenco.

Quelli che incontrerai più spesso

- `=` uguale
- `>` maggiore
- `<` minore
- `BETWEEN` dentro un intervallo
- `LIKE` testo che somiglia a un modello
- `IN` valore dentro una lista

Un esempio con un intervallo

```
SELECT *  
FROM ordini  
WHERE totale BETWEEN 50 AND 100;
```

Questa query prende gli ordini il cui totale è compreso tra 50 e 100.

LIKE : utile per il testo

Quando lavori con stringhe, `LIKE` ti aiuta a cercare un modello.

```
SELECT *  
FROM clienti  
WHERE nome LIKE 'L%';
```

Qui stai cercando nomi che iniziano con la lettera `L`.

IN : utile quando hai un piccolo elenco

Se vuoi confrontare un valore con più possibilità, `IN` rende la query più pulita:

```
SELECT *  
FROM clienti  
WHERE citta IN ('Roma', 'Milano', 'Torino');
```

Invece di scrivere tante condizioni con `OR`, metti tutto in una lista leggibile.

Il modo giusto di pensarli

Gli operatori di confronto sono il vocabolario di base dei filtri. **Più li conosci bene, più costruire condizioni diventa naturale.**

LIMIT e OFFSET

Come mostrare solo una parte dei risultati quando l'elenco è troppo lungo.

Qui impari a dosare il risultato

A volte non vuoi vedere tutte le righe. Vuoi solo le prime 5, oppure vuoi saltare le prime 10 e leggere le successive.

`LIMIT` e `OFFSET` servono proprio a questo.

```
SELECT *  
FROM clienti  
LIMIT 5;
```

Questa query restituisce solo le prime 5 righe del risultato.

Saltare le prime righe

Per saltare le prime righe:

```
SELECT *  
FROM clienti  
LIMIT 5 OFFSET 5;
```

Qui stai chiedendo: **"salta le prime 5 righe e poi mostrami le successive 5"**.

Perché è utile nella pratica

Questi strumenti sono molto usati quando costruisci:

- pagine di risultati
- liste prodotti
- tabelle amministrative
- schermate con paginazione

Un'accoppiata frequente

Spesso `LIMIT` e `OFFSET` si usano insieme a `ORDER BY`.

Per esempio, se vuoi mostrare i 10 ordini più recenti, prima ordini per data e poi limiti il risultato. **Senza ordine, il taglio delle righe può diventare poco prevedibile.**

Operatori logici

AND, OR e NOT per combinare più condizioni nella stessa query.

Quando una sola condizione non basta

Gli operatori logici servono a combinare più condizioni. Sono il modo con cui SQL ragiona su frasi come:

- clienti di Roma **e** attivi
- clienti di Roma **o** di Milano
- ordini **non** ancora spediti

```
SELECT *  
FROM clienti  
WHERE citta = 'Roma' AND attivo = TRUE;
```

AND richiede che entrambe le condizioni siano vere.

OR ne accetta almeno una. **NOT** la ribalta.

AND : tutto deve essere vero

AND funziona come una doppia porta. La riga passa solo se supera entrambe.

```
SELECT *  
FROM clienti  
WHERE citta = 'Roma' AND attivo = TRUE;
```

Qui restano solo i clienti che vivono a Roma **e** sono attivi.

OR : basta una delle due

Con **OR** la logica è più permissiva.

```
SELECT *  
FROM clienti  
WHERE citta = 'Roma' OR citta = 'Milano';
```

Qui basta che la città sia Roma oppure Milano.

NOT : capovolgere la condizione

NOT serve a negare una condizione.

```
SELECT *  
FROM clienti  
WHERE NOT attivo;
```

Questa query prende i clienti che non risultano attivi.

Perché conviene leggere la query come una frase

Quando una query ha più condizioni, il trucco migliore è leggerla ad alta voce in italiano semplice. Se la frase suona chiara, di solito anche la query è costruita bene.

Valori NULL

Come rappresentare e controllare l'assenza di un valore in SQL.

Quando un dato manca

NULL significa che un valore **non c'è** oppure **non è conosciuto**.

Non significa zero. Non significa testo vuoto. Non significa "no". Significa proprio: "qui il dato manca".

È come una casella lasciata vuota in un modulo. La casella esiste, ma nessuno ci ha scritto dentro nulla.

Cercare valori mancanti

Per trovare i clienti senza email, scrivi:

```
SELECT nome, email
FROM clienti
WHERE email IS NULL;
```

La parte importante è **IS NULL**.

Con **NULL** non si usa **=**, perché **NULL** non si comporta come un valore normale.

Questa query sembra naturale, ma è sbagliata:

```
SELECT nome, email
FROM clienti
WHERE email = NULL;
```

La forma corretta è:

```
SELECT nome, email
FROM clienti
WHERE email IS NULL;
```

In pratica non chiedi "email è uguale a mancante?". Chiedi "email è mancante?".

Cercare valori presenti

Il contrario di **IS NULL** è **IS NOT NULL**.

```
SELECT nome, email
FROM clienti
WHERE email IS NOT NULL;
```

Questa query mostra solo i clienti che hanno un'email salvata.

NULL , zero e testo vuoto

Questi tre casi sono diversi:

Valore	Significato
NULL	il dato manca
0	il dato c'è ed è zero
' '	il dato c'è ma è vuoto

Per esempio, se `sconto` è `NULL` , forse non sai ancora lo sconto. Se `sconto` è `0` , sai che lo sconto è zero.

Questa differenza è importante nei calcoli, nei filtri e nei report.

Quando lo incontrerai

`NULL` compare spesso quando:

- una colonna è facoltativa
- un dato non è stato ancora inserito
- un'informazione non è disponibile
- una `LEFT JOIN` non trova una riga collegata

Pensarlo come "dato mancante" è la scorciatoia mentale più utile.

Riepilogo rapido

- `NULL` significa dato mancante o sconosciuto.
- Non è uguale a zero.
- Non è uguale a testo vuoto.
- Si controlla con `IS NULL` .
- Il contrario si controlla con `IS NOT NULL` .

ORDER BY

Come ordinare i risultati in modo crescente o decrescente.

Dopo aver trovato i dati, puoi metterli in ordine

ORDER BY serve a scegliere l'ordine con cui il database ti mostra i risultati. Senza di lui, l'ordine non è qualcosa su cui conviene fare affidamento.

Pensa a una lista di nomi o a un elenco di prezzi. Stesso contenuto, ma un ordine diverso può renderlo molto più utile.

```
SELECT nome
FROM clienti
ORDER BY nome ASC;
```

ASC vuol dire crescente. **DESC** vuol dire decrescente.

Quando lo usi davvero

ORDER BY è utilissimo quando vuoi:

- nomi in ordine alfabetico
- prezzi dal più alto al più basso
- date dalla più recente alla più vecchia
- punteggi in classifica

Un altro esempio chiaro

```
SELECT nome, totale
FROM ordini
ORDER BY totale DESC;
```

Qui gli ordini con il totale più alto compariranno per primi.

Un dettaglio importante

L'ordinamento avviene sul risultato della query. Prima selezioni le righe, poi decidi come disporle.

Questo rende **ORDER BY** perfetto da usare dopo **WHERE**, quando hai già filtrato ciò che ti interessa.

SELECT

Come leggere i dati da una tabella scegliendo colonne e risultati utili.

Leggere dati da una tabella

SELECT è il comando che usi quando vuoi **vedere dei dati**.

Non aggiunge nulla, non cancella nulla e non modifica le informazioni salvate. Fa una cosa sola: chiede al database di mostrarti qualcosa.

Immagina una tabella `clienti` fatta così:

id	nome	email
1	Luca	luca@example.com
2	Marta	marta@example.com

Se vuoi vedere tutto quello che contiene, puoi scrivere:

```
SELECT *  
FROM clienti;
```

Il simbolo `*` significa "tutte le colonne". Quindi questa query dice: "mostrami tutto dalla tabella `clienti`".

È comoda quando stai esplorando una tabella per la prima volta. È un po' come aprire un cassetto per vedere cosa c'è dentro.

Chiedere solo quello che ti serve

Nella pratica, però, spesso non ti serve tutto. Magari vuoi vedere solo il nome e l'email dei clienti:

```
SELECT nome, email  
FROM clienti;
```

Il risultato diventa più pulito:

nome	email
Luca	luca@example.com
Marta	marta@example.com

Hai ancora gli stessi clienti, ma stai guardando solo le colonne utili per la tua domanda.

Questa è una buona abitudine: più la query è precisa, più è facile da leggere.

Come leggere una query **SELECT**

Una query di lettura molto semplice ha due parti:

- **SELECT** dice **quali colonne** vuoi vedere
- **FROM** dice **da quale tabella** prenderle

Questa query:

```
SELECT nome, email  
FROM clienti;
```

si può leggere così:

*"Vai nella tabella **clienti** e mostrami le colonne **nome** ed **email**."*

Quando sei agli inizi, tradurre la query in italiano aiuta molto. Se la frase italiana suona confusa, probabilmente anche la query merita di essere riscritta meglio.

Perché non usare sempre **SELECT ***

SELECT * va benissimo per curiosare in una tabella piccola o per fare prove veloci.

Quando però scrivi una query che vuoi conservare, condividere o usare in un'applicazione, è meglio indicare le colonne per nome:

```
SELECT nome, email
FROM clienti;
```

Così chi legge capisce subito quali dati ti interessano. Inoltre eviti di portarti dietro colonne inutili, magari pesanti o sensibili.

Un esempio con i prodotti

Se hai una tabella `prodotti` con colonne `id`, `nome`, `prezzo` e `disponibile`, e vuoi vedere solo nome e prezzo, scrivi:

```
SELECT nome, prezzo
FROM prodotti;
```

Il gesto è sempre lo stesso: scegli la tabella, poi scegli le colonne.

Riepilogo rapido

- `SELECT` legge dati.
- `FROM` indica la tabella.
- `*` significa "tutte le colonne".
- Nelle query curate, è meglio indicare solo le colonne che servono.

WHERE

Come filtrare le righe che ti servono davvero, senza leggere tutto.

Scegliere quali righe vedere

`WHERE` serve a dire al database: **mostrami solo le righe che rispettano questa condizione.**

Senza `WHERE`, una query legge tutte le righe della tabella. Con `WHERE`, invece, restringi il risultato.

È come cercare in una rubrica solo le persone che vivono a Roma, invece di leggere tutti i contatti uno per uno.

```
SELECT nome, citta
FROM clienti
WHERE citta = 'Roma';
```

Questa query significa:

*“Mostrami nome e città dei clienti che hanno **Roma** nella colonna **citta** .”*

Se nella tabella ci sono clienti di città diverse, vedrai solo quelli di Roma.

La condizione dopo **WHERE**

La parte importante è questa:

```
WHERE citta = 'Roma'
```

SQL controlla la condizione una riga alla volta.

Se la riga ha **Roma** nella colonna **citta** , entra nel risultato. Se ha **Milano** , **Napoli** o qualsiasi altro valore, viene esclusa.

Nota: I testi si scrivono di solito tra apici singoli, come **'Roma'** . I numeri invece si scrivono senza apici, come **18** .

Un esempio con i numeri

Ora immaginiamo una tabella **ordini** :

id	totale
1	35.00
2	120.00
3	89.90

Per vedere solo gli ordini sopra 100, scrivi:

```
SELECT id, totale
FROM ordini
WHERE totale > 100;
```

Il risultato sarà:

id	totale
2	120.00

La query non ha “saltato” le altre righe per caso. Le ha controllate e ha tenuto solo quella in cui `totale > 100` era vero.

Gli operatori più comuni

Dopo `WHERE` puoi usare diversi operatori:

Operatore	Significato	Esempio
<code>=</code>	uguale a	<code>citta = 'Roma'</code>
<code><></code>	diverso da	<code>stato <> 'annullato'</code>
<code>></code>	maggiore di	<code>totale > 100</code>
<code><</code>	minore di	<code>eta < 18</code>
<code>>=</code>	almeno	<code>prezzo >= 10</code>
<code><=</code>	al massimo	<code>quantita <= 5</code>

Non serve impararli tutti a memoria subito. Li userai così spesso che diventeranno familiari.

Testi, numeri e date

Il valore che confronti deve essere scritto nel modo giusto.

Per un testo usi gli apici:

```
SELECT nome
FROM clienti
WHERE citta = 'Roma';
```

Per un numero no:

```
SELECT id, totale
FROM ordini
WHERE totale >= 50;
```

Per una data si usa spesso il formato `AAAA-MM-GG` :

```
SELECT id, data_ordine
FROM ordini
WHERE data_ordine >= '2026-01-01';
```

Il database capisce che quel testo rappresenta una data se la colonna `data_ordine` è stata creata con un tipo adatto.

Una distinzione importante

Con `SELECT` , `WHERE` filtra solo quello che vedi. Non cambia i dati.

Ma più avanti userai `WHERE` anche con comandi come `UPDATE` e `DELETE` . Lì diventa molto più delicato, perché decide quali righe modificare o cancellare.

Attenzione: Prima di modificare o cancellare dati, controlla sempre il filtro con una `SELECT` . È una piccola abitudine che evita grandi guai.

Riepilogo rapido

- `WHERE` filtra le righe.
- La condizione può essere vera o falsa.
- SQL controlla la condizione riga per riga.
- Con `UPDATE` e `DELETE`, un `WHERE` sbagliato può toccare troppe righe.